Efficient Models Training

Applied AI Skoltech

Shvetsov Egor Head of Research group

How to Train models faster?

CATEGORY I Hardware-Aware Optimizations Ни как не меняем внутреннюю часть моделей, ни как не влияет на веса модели. Обучаем в точности туже модель, но быстрее (в пределах вычислительной точности).	Compilation & Custom Kernels (Flash Attention): 10-30% Memory & I/O: 5- 10% Distributed Training: 10- 30% Data Pipeline: 20 - 40%	**Universal and task agnostic strategy**
CATEGORY II Model-Intrinsic Optimizations Поменяем архитектуру модели и то как она обучается. Другими словам мы получим немного другую модель, но которая обучается решать туже задачу быстрее.	Optimizers: 10 - 50% Learning Rate Schedules: 10 -20% Initialization: 10 - 50% Architecture Modifications: 10 - 30% Data Efficiency & Augmentation	**Highly task specific**
CATEGORY III Computational approximations Что-то среднее между I и II. Немного меняем данные - аугментируем, чуть занулуяем и так маленькие по магнитуде веса.	Quantization & Sparsification: 5 - 20% (sometimes up to 90%) Batching 10 - 20%	**Somewhere in the middle**

NanoGPT Speedrun

NanoGPT Speedrun Setup

Sourse: https://github.com/kellerjordan/modded-nanogpt

Task: Train GPT-2 (124M params) to **3.28 validation CE loss** on FineWeb dataset

Baseline: Karpathy's NanoGPT — 8.13h on 2×4090, ~45 min on 8×H100

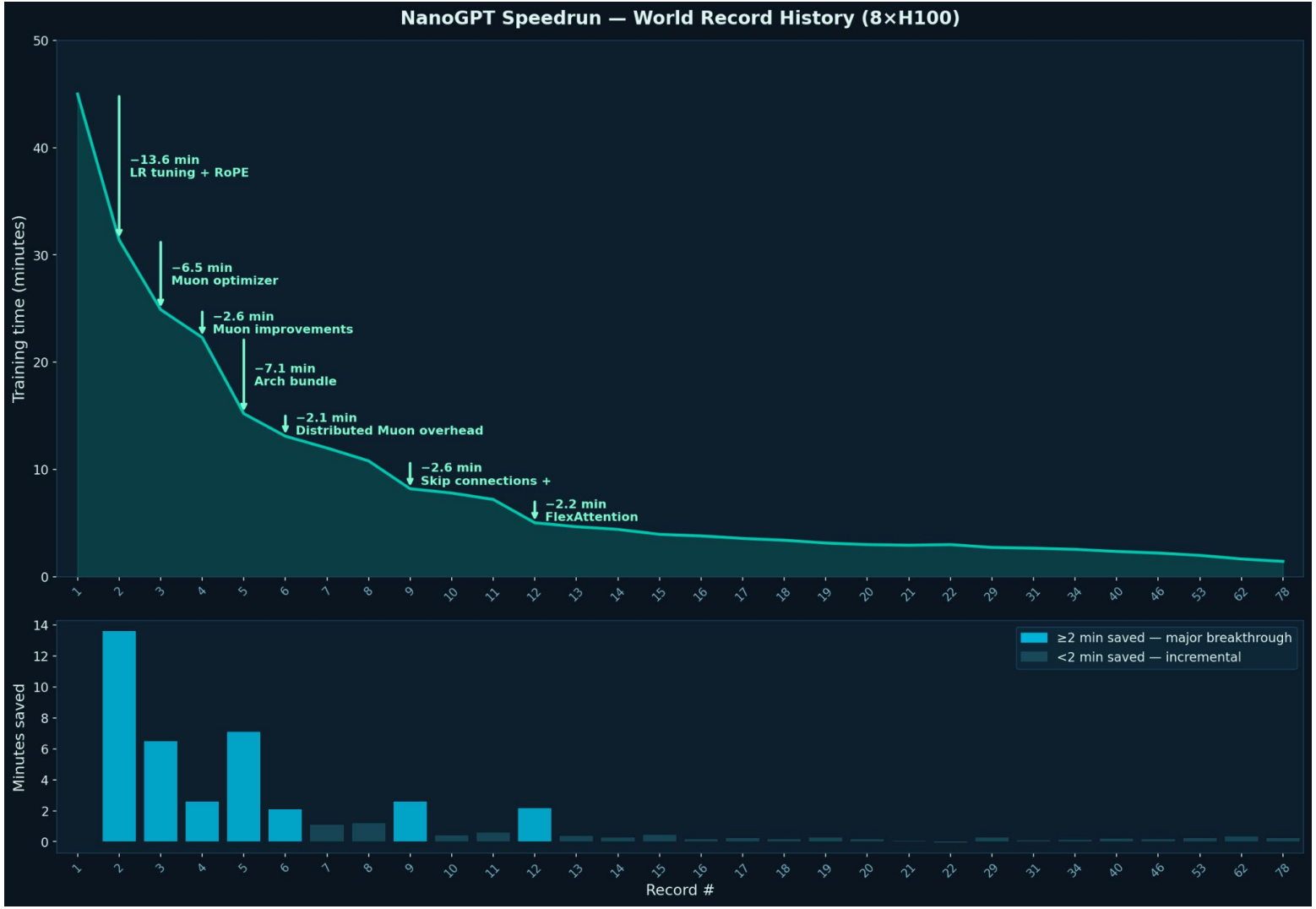
Results for 2× RTX 4090

Step	Time	Improvement	What changed
Baseline	8.13h	—	Karpathy's NanoGPT, standard AdamW, 1024 token sequences
Architectural changes	7.51h	**−8% time**	RoPE, ReLU², RMSNorm, trapezoidal LR, no gradient clipping
Muon optimizer	4.53h	**−40% time**	Replace AdamW with SGD + gradient orthogonalization for Linear layers
Dataloading tweaks	4.26h	**−6% time**	Load only next microbatch per step, PyTorch 2.6 upgrade
Logit soft-capping	4.01h	**−6% time**	cap × tanh(x/cap) prevents logit explosion, stabilizes small model training
Longer sequences	2.55h	**−36% time**	FlexAttention with document masking, 1k → 32k token sequences

NanoGPT Speedrun

Official RUN: 8×H100

68	1.535 minutes	Move bigram hash to GPU	01/31/26	log,PR	@dhrvji
69	1.528 minutes	Kernel Optimizations	02/02/26	log,PR	@EmmettBicker & AI System Aster
70	1.521 minutes	Tune value embed layout and ve_gates	02/03/26	log,PR	@photon_mz
71	1.516 minutes	Sparse bigram gradient comms and optimized loading on CPU	02/06/26	log,PR	@roeeshenberg
72	1.496 minutes	Increase minimum lr and add max_seq_len schedule	02/10/26	log,PR	@dualverse-ai & AI System Station
73	1.485 minutes	Partitioned Hyperconnections	02/12/26	log,PR	@sisovicm
74	1.468 minutes	Flattened GPT forward, removed post attention lambdas, added transpose kernels	02/16/26	log,PR	@ChrisJMcCormick
75	1.453 minutes	Cross Entropy Kernel Optimizations	02/23/26	log,PR	@moof2x
76	1.446 minutes	Reuse and tune backward transpose kernel	02/28/26	log,PR	@samacqua
77	1.435 minutes	Replace partitioned hyperconnections with single saved activation	03/06/26	log,PR	@classiclarryd
78	1.426 minutes	Tighten bounds on fa3 max_num_docs to match fineweb distribution	03/22/26	log,PR	@ChrisJMcCormick


NanoGPT Speedrun — World Record History (8×H100)
Training time (minutes)
−13.6 min
LR tuning + RoPE
−6.5 min
Muon optimizer
−2.6 min
Muon improvements
−7.1 min
Arch bundle
−2.1 min
Distributed Muon overhead
−2.6 min
Skip connections +
−2.2 min
FlexAttention
Minutes saved
≥2 min saved — major breakthrough
<2 min saved — incremental
Record #

NanoGPT Speedrun

NanoGPT Speedrun Setup

Sourse: https://github.com/kellerjordan/modded-nanogpt

Task: Train GPT-2 (124M params) to **3.28 validation CE loss** on FineWeb dataset

Baseline: Karpathy's NanoGPT — 8.13h on 2×4090, ~45 min on 8×H100

Official RUN: 8×H100

Rank	Technique	Saved	% reduction
1	LR tuning + RoPE	−13.6 min	−30%
2	Arch bundle (ReLU², QK-norm, zero-init)	−7.1 min	−32%
3	Muon optimizer	−6.5 min	−21%
4	Skip connections + logit softcap	−2.6 min	−24%
5	Muon improvements	−2.6 min	−10%
6	Distributed Muon overhead	−2.1 min	−14%
7	FlexAttention (1k→64k context)	−2.2 min	−30%

NanoGPT Speedrun

Muon - Keller Jordan https://kellerjordan.github.io/posts/muon/

The Problem: Skewed Singular Value Distributions:

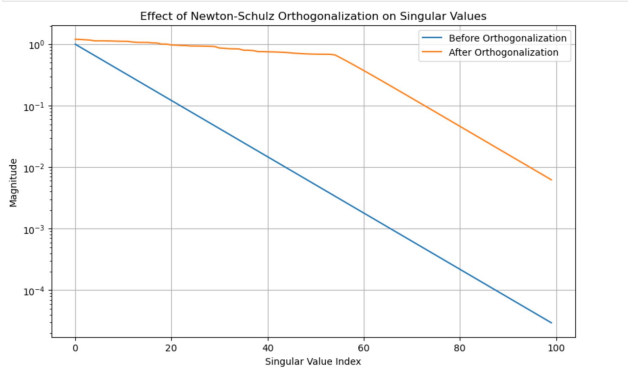
- A few singular values are very large
- Most singular values are extremely small
- This skewed distribution means we're effectively only learning in a small subspace

Muon's Solution: Matrix Orthogonalization

- Muon addresses this problem through a technique called matrix orthogonalization. For matrix parameters, Muon:
- Computes standard momentum updates
- Orthogonalizes these updates using Newton-Schulz iteration
- Applies the orthogonalized updates to the parameters

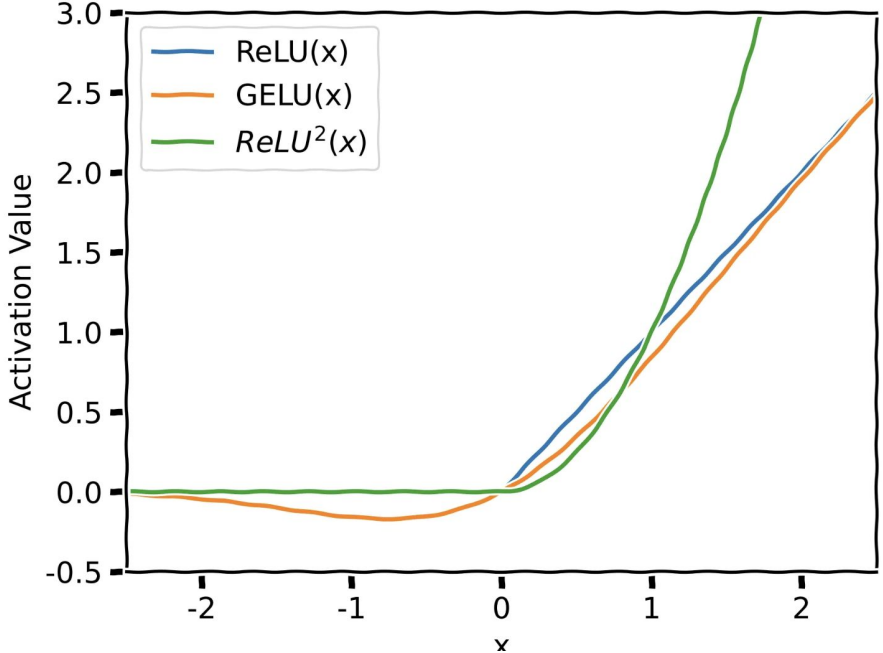
After orthogonalization:

- Singular values become more uniform
- Updates act across all directions in parameter space
- The neural network can utilize its full parameter capacity



GOOD for Conv layers and linear layers, does not work for scalars.

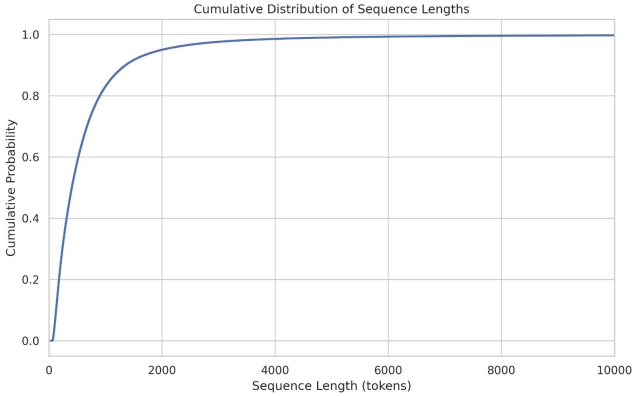
NanoGPT Speedrun



NanoGPT Speedrun

Longer sequences

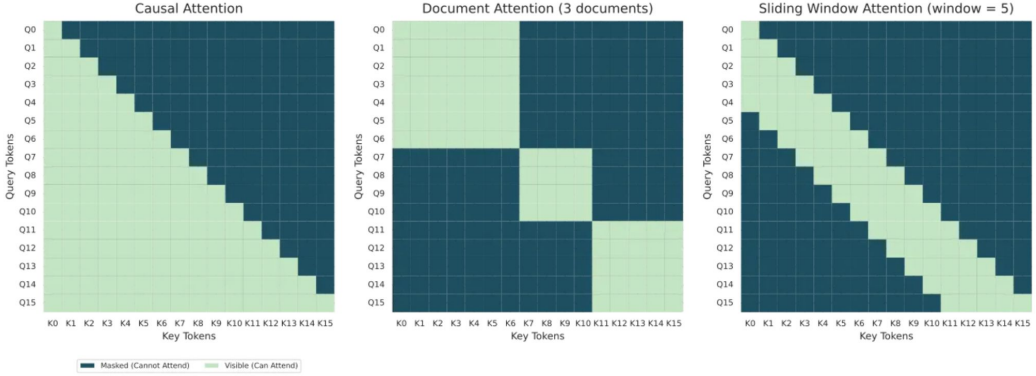
- The model is trained on FineWeb, sequences are longer than 1024 tokens.
- Cutting sequences in the middle is not good.
- If we use 8192 we will fot almost all lengths.



Proposed solution:

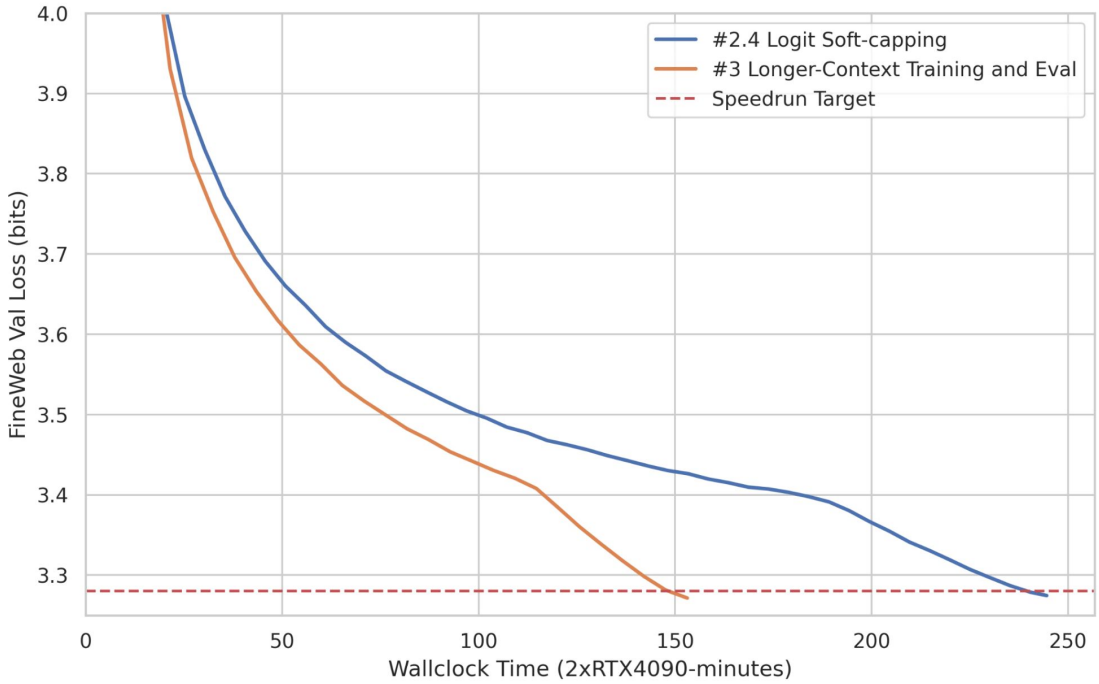
- Use batch size 1;
- Concatenate all sequences into one long document;
- Add cross document attention Masking since we have many documents right now with FlexAttention.

For each batch size 1 we construct a block wise mask.



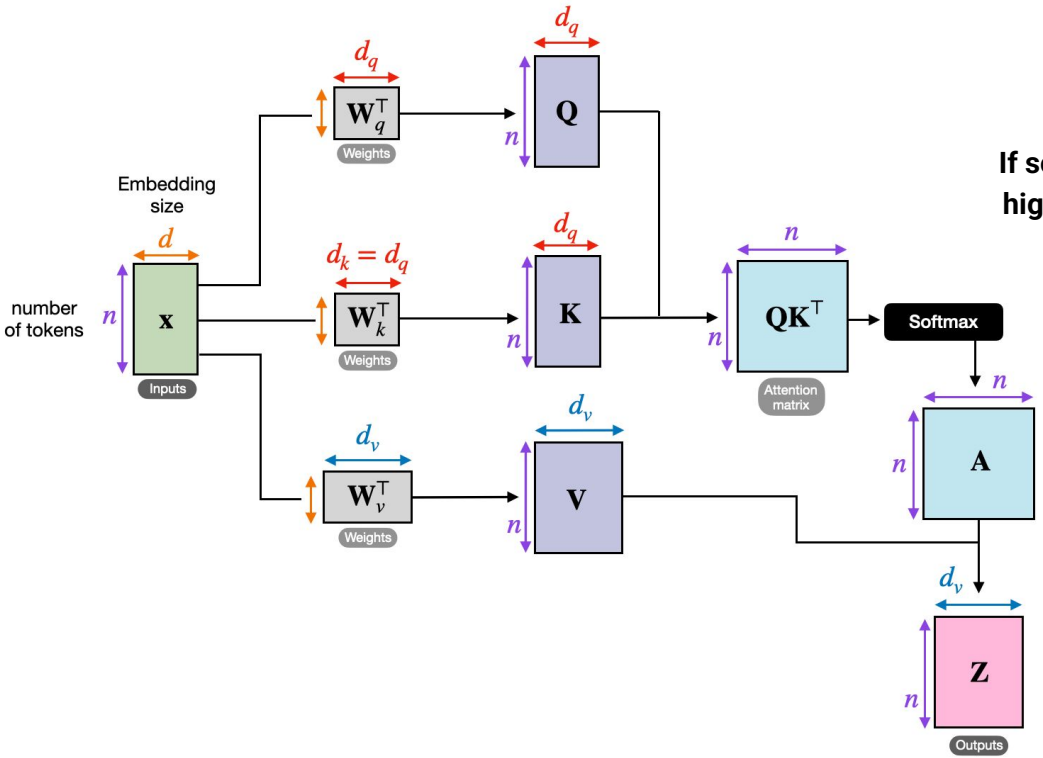
NanoGPT Speedrun

Longer sequences



NanoGPT Speedrun

Attention Logit Explosion : Self Attention Recap



If some values of QT^T are high Softmas will zero out other values

NanoGPT Speedrun

Attention Logit Explosion : Solution

$$a_{ij} = \frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_k}}$$

$$\mathbf{q}^\top \mathbf{k} \sim |\mathbf{q}| \cdot |\mathbf{k}| \cdot \cos\theta$$

- Attention growth almost quadratically in magnitude.
- If one value is very high then followed by SfotMax some tokens may get low gradient values.

Let's normalize values:

1) before dot product: $\hat{\mathbf{q}} = \frac{\mathbf{q}}{|\mathbf{q}|}, \qquad \hat{\mathbf{k}} = \frac{\mathbf{k}}{|\mathbf{k}|}$

$$a_{ij} = \frac{\hat{\mathbf{q}}_i^\top \hat{\mathbf{k}}_j}{\sqrt{d_k}} \in \left[-\frac{1}{\sqrt{d_k}}, \frac{1}{\sqrt{d_k}}\right]$$

2) after dot product: $a_{ij} = c \cdot \tanh\left(\frac{\mathbf{q}_i^\top \mathbf{k}_j}{c}\right)$

NanoGPT Speedrun

Easy Path with skips : Solution

```python
# GPT BLOCK
class Block(nn.Module):

    def __init__(self, config):
        super().__init__()
        self.attn = CausalSelfAttention(config)
        self.mlp = MLP(config)
        self.attn_scale = 1 / math.sqrt(2 * config.n_layer)
        self.norm_one = RMSNorm(config.n_embd)
        self.norm_two = RMSNorm(config.n_embd)

    def forward(self, x):
        x = σ(u)*x + self.attn_scale * self.attn(self.norm_one(x))
        x = σ(u)*x + self.mlp(self.norm_two(x))
        return x
```

Gamma with a sigmoid gate that is initialized to produce 0.18.

$\sigma(u) = 0.18$, u is trainable.

The smaller initialization for skip connections gives the model worse initial training, but better final performance due to encouraging the formation of deeper pathways.

CIFAR-10 in 3.29 Seconds

Setup:

Single NVIDIA A100

Target 94% accuracy

Model: 1.97M parameter VGG-like network (ResNet-18: ~11.7M). It contains 7 convolutions with the latter six being divided into

three blocks of two.

Rank	Technique	Before → After	Saved	% reduction
1	Frozen patch-whitening init	18.3s → 8.0s	−10.3s	−56%
2	Scale BN biases (64× LR)	6.8s → 5.1s	−1.7s	−25%
3	DIRAC identity init	8.0s → 6.8s	−1.2s	−15%
4	torch.compile	3.83s → 3.29s	−0.54s	−14%
5	Lookahead optimizer	5.1s → 4.6s	−0.5s	−10%
6	Multi-crop TTA	4.6s → 4.2s	−0.4s	−9%
7	Alternating flip	4.2s → 3.83s	−0.37s	−9%

CIFAR-10 in 3.29 Seconds

arxiv.org/abs/2501.17116

CIFAR-10 in 3.29 Seconds

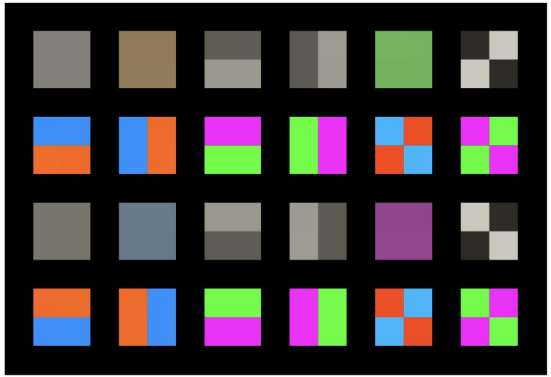
Patch - whitening init - almost 2 times faster:

- The first 12 filters are initialized as the eigenvectors of the covariance matrix of 2x2 patches across the training distribution, so that their outputs have identity covariance matrix.

- The second 12 filters are initialized as the negation of the first 12, so that input information is preserved through the activation which follows.

So, it is a normalization layer encoded into the weighs!

	Removes mean	**Normalizes variance**	**Removes correlations**
BatchNorm	Yes	Yes (per channel)	No
Patch whitening	Yes	Yes	Yes



First 12 channels

CIFAR-10 in 3.29 Seconds

Patch - whitening init - almost 2 times faster:

1. Extract and center patches $\mathbf{x} \in \mathbb{R}^d, \quad d = 2 \times 2 \times 3 = 12$

$$\tilde{\mathbf{x}} = \mathbf{x} - \boldsymbol{\mu}, \qquad \boldsymbol{\mu} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{x}_i$$

2. Covariance matrix

$$\mathbf{C} = \frac{1}{N}\widetilde{\mathbf{X}}\widetilde{\mathbf{X}}^\top \in \mathbb{R}^{d\times d}, \qquad C_{ij} = \mathbb{E}[\tilde{x}_i\,\tilde{x}_j]$$

3. Eigendecomposition

$$\mathbf{C} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\top, \qquad \boldsymbol{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_d), \quad \lambda_1 \geq \cdots \geq \lambda_d \geq 0$$

4. Whitening transform

$$\mathbf{W} = \boldsymbol{\Lambda}^{-1/2}\mathbf{V}^\top, \qquad \boldsymbol{\Lambda}^{-1/2} = \operatorname{diag}\left(\frac{1}{\sqrt{\lambda_i + \varepsilon}}\right), \quad \varepsilon = 5 \times 10^{-4}$$

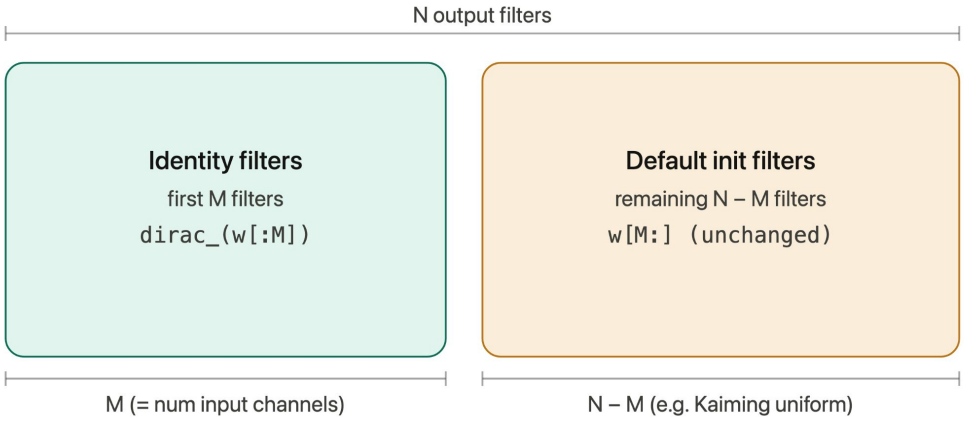
5. Apply & verify

$$\mathbf{z} = \mathbf{W}\tilde{\mathbf{x}}$$

$$\mathbb{E}[\mathbf{z}\mathbf{z}^\top] = \mathbf{W}\mathbf{C}\mathbf{W}^\top = \boldsymbol{\Lambda}^{-1/2}\mathbf{V}^\top \cdot \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\top \cdot \mathbf{V}\boldsymbol{\Lambda}^{-1/2} = \boldsymbol{\Lambda}^{-1/2}\boldsymbol{\Lambda}\boldsymbol{\Lambda}^{-1/2} = \mathbf{I} \quad \checkmark$$

CIFAR-10 in 3.29 Seconds

DIRAC identity init:



Each identity filter copies one input channel exactly (1 at center, 0 elsewhere)

→ network starts as near-identity, then learns deviations from it

CIFAR-10 in 3.29 Seconds

Scaled BN - Increase the learning rate for the learnable biases of all BatchNorm layers by a factor of 64, increase weight decay.

Assumption: BN parameters are stable during training but are updated much slower. Faster updates lead to faster and more stable convergence.

Training acceleration via Quantization

Paper	Speed up
Jetfire: Efficient and Accurate Transformer Pretraining with INT8 Data Flow and Per-Block Quantization	25%
Stable and low-precision training for large-scale vision-language models	20%

Quantization

FP32 tensor $\boldsymbol{X} \approx s_X \boldsymbol{X}_{\text{int}} = \hat{\boldsymbol{X}}$ scaled quantized tensor

$$\boldsymbol{W} = \begin{pmatrix} 0.97 & 0.64 & 0.74 & \boxed{1.00} \\ 0.58 & 0.84 & 0.84 & 0.81 \\ \boxed{0.00} & 0.18 & 0.90 & 0.28 \\ 0.57 & 0.96 & 0.80 & 0.81 \end{pmatrix} \approx \frac{1}{255} \begin{pmatrix} 247 & 163 & 189 & \boxed{255} \\ 148 & 214 & 214 & 207 \\ \boxed{0} & 46 & 229 & 71 \\ 145 & 245 & 204 & 207 \end{pmatrix} = s_W \boldsymbol{W}_{\text{uint8}}$$

Quantization is cheap but not free!

$$Q_{\text{row}}\left(\begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}\right) = \text{round}\left(\begin{bmatrix} \frac{127}{\text{absmax}(x_1)} \cdot x_1 \\ \vdots \\ \frac{127}{\text{absmax}(x_b)} \cdot x_b \end{bmatrix}\right)$$

Accelerating MatMul with quantization

Batch Size	q_proj	k_proj	v_proj	o_proj	gate_proj	up_proj	down_proj	Overall Speedup (%)
4	-320.17	-1844.99	-1800.91	-490.18	-86.64	-84.61	-62.26	-225.68
8	-234.68	-1205.47	-1565.68	-182.08	-32.27	-32.51	-61.97	-124.02
16	-44.91	-757.88	-752.47	-88.79	-10.96	-14.16	-21.16	-50.71
32	11.88	-541.97	-469.65	7.14	25.61	24.93	8.41	3.94
64	24.41	-164.54	-166.06	23.17	31.12	32.52	20.96	22.49
128	28.96	-36.32	-57.08	28.47	31.00	30.60	16.24	24.01
256	31.40	-43.24	-48.99	30.99	29.98	30.98	19.14	25.42
512	31.48	-53.22	-54.34	29.62	33.34	32.52	23.37	27.62

LAYER	SIZE
q_proj	**2048:2048**
k_proj	2048:256
v_proj	2048:256
o_proj	**2048:2048**
gate_proj	**2048:5632**
up_proj	**2048:5632**
down_proj	**5632:2048**

It all depends on a batch size and layer size

Quantization is better for speed!

Matrix Multiplication	Input Size	Weight Size	Time	SpeedUp
Dense fp16	(32*100,256)	(887045, 256)	34.4769s	-
Semi-structural 2:4	(32*100,256)	(887072, 256)	22.9272s	1.504
Quantization W8A8-INT	(32*100,256)	(887072, 256)	14.8956	2.315

Model Training Acceleration directions

Forward: $\mathbf{Y} = \mathbf{X}\mathbf{W} + \mathbf{b}$

Backward: $\partial \mathcal{L} / \partial \mathbf{Y}$:

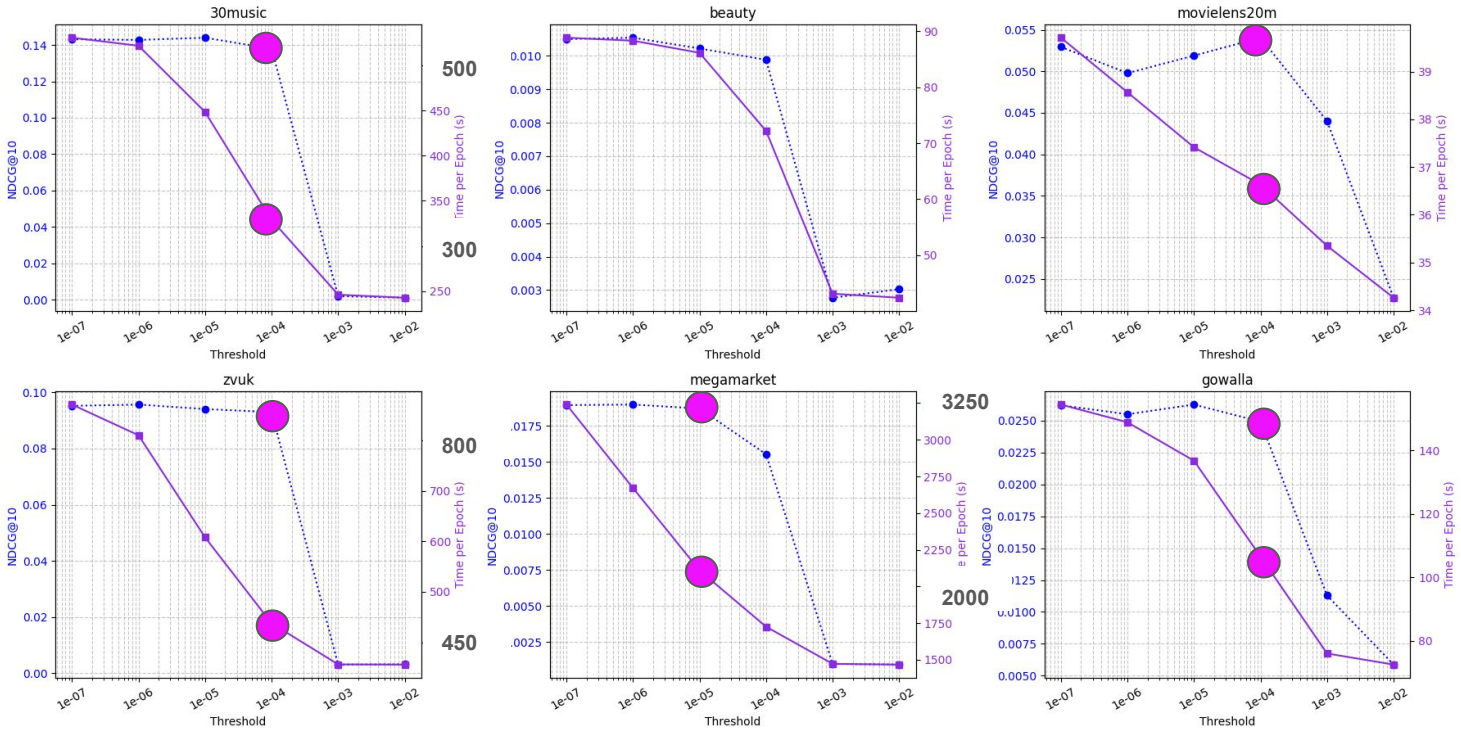
Input gradient: $\frac{\partial \mathcal{L}}{\partial \mathbf{X}} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{Y}}\right) \mathbf{W}^\top$

Weight gradient: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{X}^\top \left(\frac{\partial \mathcal{L}}{\partial \mathbf{Y}}\right)$

Bias gradient: $\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \sum \text{rows of } \frac{\partial \mathcal{L}}{\partial \mathbf{Y}}$

~ two times more FLOPs

Training Acceleration with gradient pruning



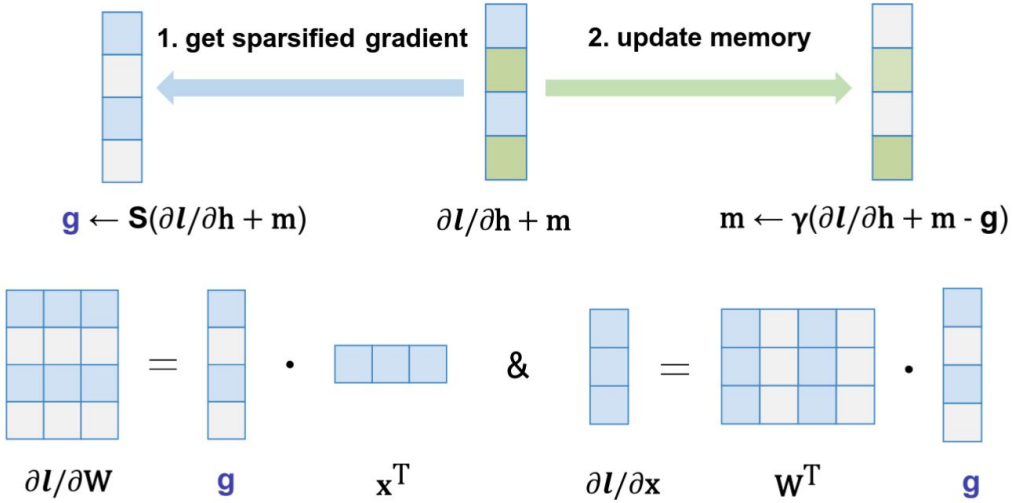
Acceleration up to 1.5 times

Can we do better?

Better Sparse Gradients

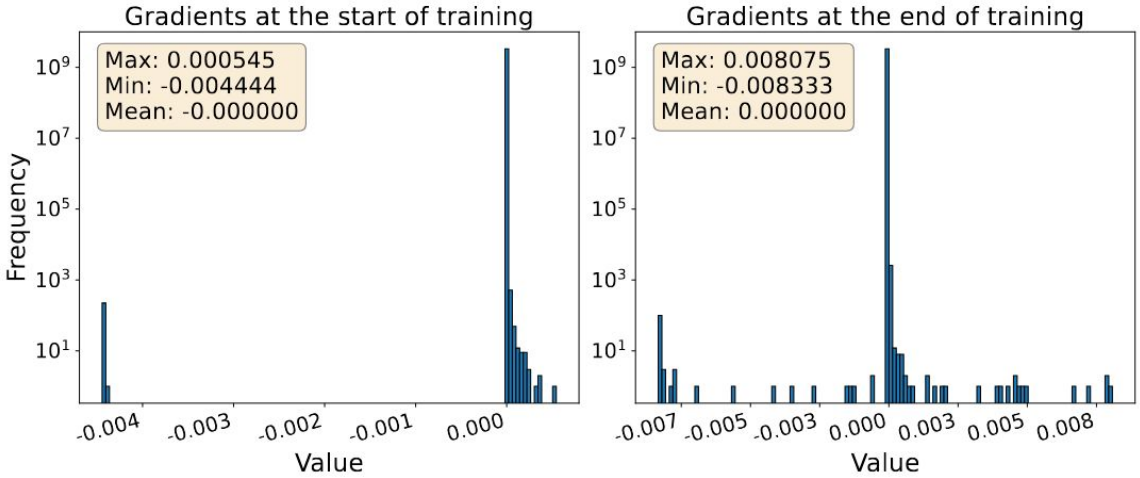
Algorithm 2 Memorized sparse backpropagation learning for a linear layer

Initialize learnable parameter $\mathbf{W}$
Initialize gradient memory $\mathbf{m} \leftarrow \mathbf{0}$
while training **do**
  /* forward */
  Get input of this layer $\mathbf{x}$
  $\mathbf{h} \leftarrow \mathbf{W}\mathbf{x}$
  $\mathbf{z} \leftarrow \sigma(\mathbf{h})$
  Propagate $\mathbf{z}$ to the next layer
  /* backward */
  Get $\frac{\partial \ell}{\partial \mathbf{z}}$ propagated from the next layer
  $\frac{\partial \ell}{\partial \mathbf{h}} \leftarrow \sigma'(\mathbf{h}) \odot \frac{\partial \ell}{\partial \mathbf{z}}$
  $\mathbf{g} \leftarrow S_{\mathbb{I}_k}(\frac{\partial \ell}{\partial \mathbf{h}} + \mathbf{m})$
  $\mathbf{m} \leftarrow \gamma(\frac{\partial \ell}{\partial \mathbf{h}} + \mathbf{m} - \mathbf{g})$
  $\frac{\partial \ell}{\partial \mathbf{W}} \leftarrow \mathbf{g}\mathbf{x}^{\mathrm{T}}$
  $\frac{\partial \ell}{\partial \mathbf{x}} \leftarrow \mathbf{W}^{\mathrm{T}}\mathbf{g}$
  Backpropagate $\frac{\partial \ell}{\partial \mathbf{x}}$ to the previous layer
  /* update */
  Update $\mathbf{W}$ with $\frac{\partial \ell}{\partial \mathbf{W}}$
end while



Problems with this approach?

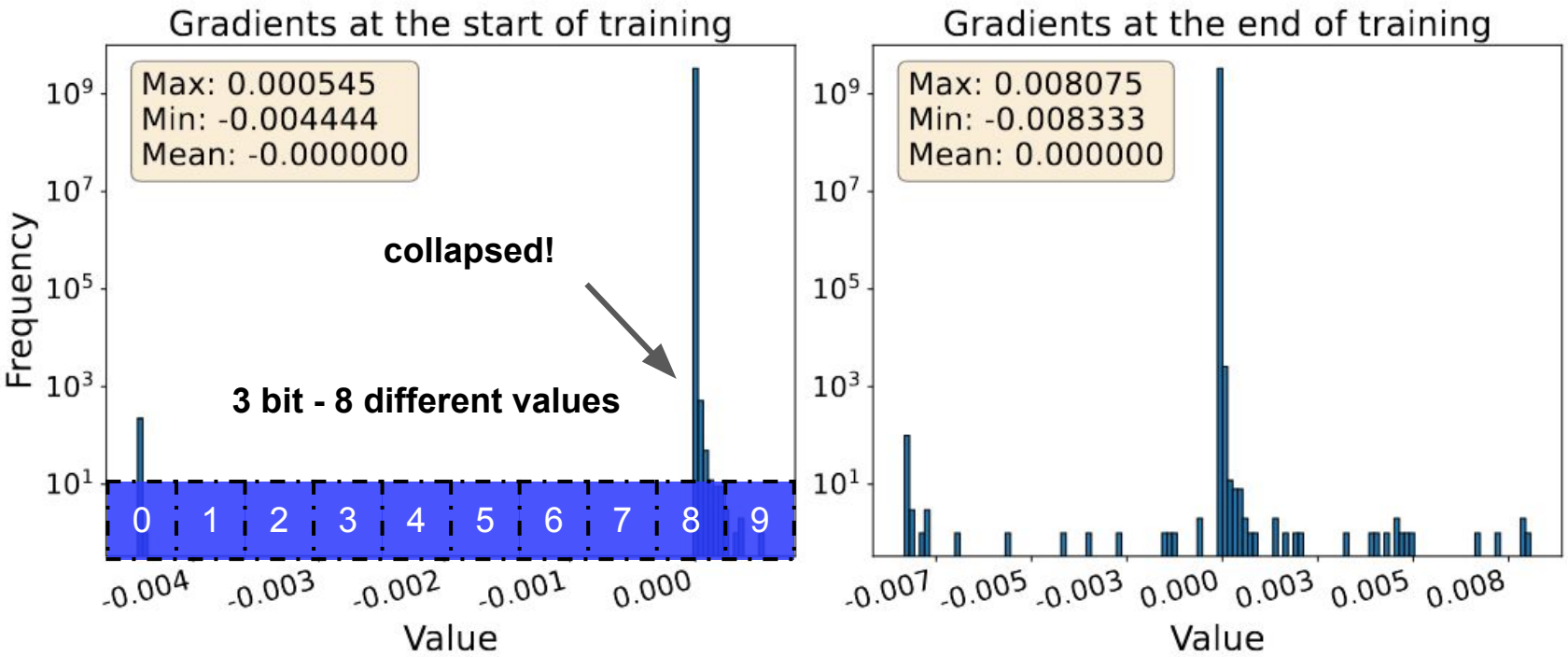
Quantization of gradients



Potential problems?

$$Q_{\text{row}}\left(\begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}\right) = \text{round}\left(\begin{bmatrix} \frac{127}{\text{absmax}(x_1)} \cdot x_1 \\ \vdots \\ \frac{127}{\text{absmax}(x_b)} \cdot x_b \end{bmatrix}\right)$$

Quantization of gradients



What can we do here?

- **Compute percentile and remove outlier?**

What can we do here?

- **Compute percentile and remove outlier?**
- **But it takes too much time!**

What can we do here?

- **Compute percentile and remove outlier?**
- **But it takes too much time!**

What we have so far?

- Quantization gives better acceleration than sparsification;
- Actual acceleration depends on a matrix size (sequence length x batch size etc.);
- Actual acceleration depends on hardware type - what works on A40 - does not work A100.
- Both quantization and sparsification during training cause computational overhead
- It is possible to sparsify gradients!
- Most ot the papers focus on only forward path quantization during training

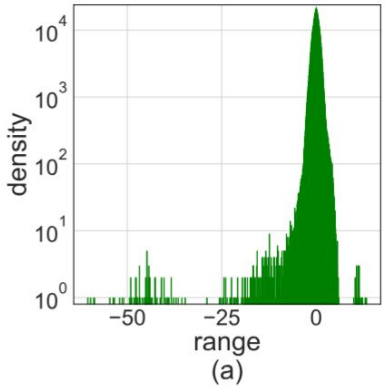
What else?

Gradient Quantization

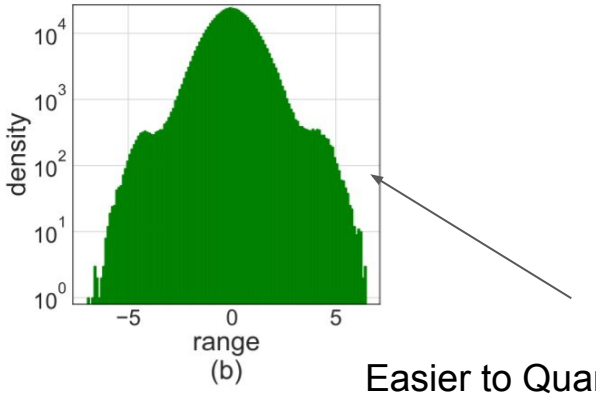
Procedure `HQ-MM`

1. Compute $\mathbf{XH}$ and $\mathbf{H}^\top \mathbf{W}^\top$ in FP16.
2. Quantize the resultant matrices to INT4 by LSQ.
3. Multiply the two INT4 matrices.
4. Dequantize the resultant INT32 matrix to FP16 by multiplying $s_X s_W$.

Original

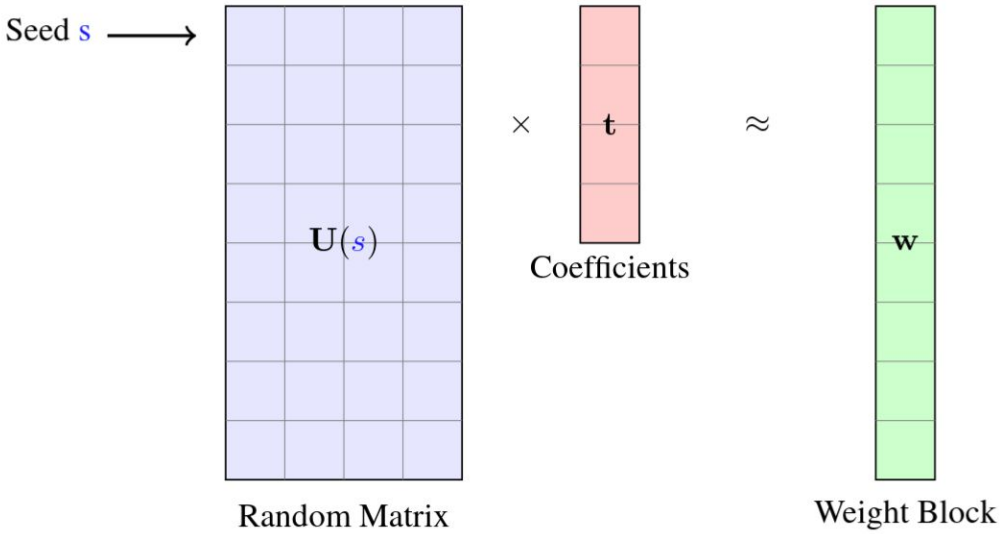


Hadamar Transformed



Easier to Quantize

Just to Fun



[SeedLM: Compressing LLM Weights into Seeds of Pseudo-Random Generators]

LITERATURE:

- Accelerating CNN Training by Pruning Activation Gradients
- The Convergence of Sparsified Gradient Methods
- meProp: Sparsified Back Propagation for Accelerated Deep Learning with Reduced Overfitting
- DEFT: Exploiting Gradient Norm Difference between Model Layers for Scalable Gradient Sparsification
- MAST: Model-Agnostic Sparsified Training
- Sparse Weight Activation Training
- Accelerating Deep Neural Networks via Semi-Structured Activation Sparsity
- Always-Sparse Training by Growing Connections with Guided Stochastic Exploration
- Dual sparse training framework: inducing activation map sparsity via Transformed ℓ1 regularization
- PruneTrain: Fast Neural Network Training by Dynamic Sparse Model Reconfiguration
- Memorized Sparse Backpropagation
- HOT: Hadamard-based Optimized Training
- Jetfire: Efficient and Accurate Transformer Pretraining with INT8 Data Flow and Per-Block Quantization
- Stable and low-precision training for large-scale vision-language models
- Oscillation-reduced mxfp4 training for vision transformers
- Accurate int8 training through dynamic block-level fallback
- Towards efficient pre-training: Exploring fp4 precision in large language models
- Quartet: Native FP4 Training Can Be Optimal for Large Language Models